Agenda

- Interfejsy funkcyjne
  - Funkcje lambda
- Strumienie

Interfejsy funkcyjne

Wyświetlenie danych z pliku

```java
public class FileCommander {
  public static void cat(String filePath) {
    try (FileReader fr = new FileReader(filePath)) {
      String line;
      while ((line = fr.readLine()) != null) {
        System.out.println(line);
      }
    } catch (IOException _) { }
  }
}

FileCommander.cat("src/FileCommander.java");
// public class FileCommander {
// ...
```

Funkcja modyfikująca napis

```
public static String switchLetterCase(String str) {
  StringBuilder builder = new StringBuilder();
  for (char c : str.toCharArray()) {
    if (Character.isUpperCase(c))
      builder.append(Character.toLowerCase(c));
    else if (Character.isLowerCase(c))
      builder.append(Character.toUpperCase(c));
    else
      builder.append(c);
  }
  return builder;
}
```

Modyfikacja przed wyświetleniem

CamelCase

↓

cAMELcASE

```
public class FileCommander {
  public static void cat(String filePath) {
    try (BufferedReader fr = new BufferedReader(
                                 new FileReader(filePath))) {
      String line;
      while ((line = fr.readLine()) != null) {
        line = switchLetterCase(line);
        System.out.println(line);
      }
    } catch (IOException _) { }
  }
}
FileCommander.cat("src/FileCommander.java");
// PUBLIC CLASS fILE cOMMANDER {
// ...
```

Interfejs funkcyjny

```
public class Main {
  public static Output doSomething(Input input) {
    /* ... */
  }

  public static void main(String[] args) {
    Function<Input, Output> action = Main::doSomething;
    Input input = new Input();
    Output output = action.apply(input);
  }
}
```

Interfejs funkcyjny

```
public class FileCommander {
  public static void cat(String filePath, Function<String, String> action) {
    try (BufferedReader fr = new BufferedReader(new FileReader(filePath))) {
      String line;
      while ((line = fr.readLine()) != null) {
        line = action.apply(line);
        System.out.println(line);
      }
    } catch (IOException _) {}
  }
}
FileCommander.cat("src/FileCommander.java", FileCommander::switchLetterCase);
// PUBLIC CLASS FILE COMMANDER {
// ...
```

Przywrócenie poprzedniej funkcjonalności

```
public static String doNothing(String line) {
  return line;
}

FileCommander.cat(
    "src/FileCommander.java",
    FileCommander::doNothing
);

// public class FileCommander {
// ...
```

Package java.util.function

package java.util.function

Interfaces

Class	Description
BiConsumer<T,U>	Represents an operation that accepts two input arguments and returns no result.
BiFunction<T,U,R>	Represents a function that accepts two arguments and produces a result.
BinaryOperator<T>	Represents an operation upon two operands of the same type, producing a result of the same type as the operands.
BiPredicate<T,U>	Represents a predicate (boolean-valued function) of two arguments.
BooleanSupplier	Represents a supplier of `boolean`-valued results.
Consumer<T>	Represents an operation that accepts a single input argument and returns no result.
DoubleBinaryOperator	Represents an operation upon two `double`-valued operands and producing a `double`-valued result.
DoubleConsumer	Represents an operation that accepts a single `double`-valued argument and returns no result.
DoubleFunction<R>	Represents a function that accepts a double-valued argument and produces a result.
DoublePredicate	Represents a predicate (boolean-valued function) of one `double`-valued argument.
DoubleSupplier	Represents a supplier of `double`-valued results.
DoubleToIntFunction	Represents a function that accepts a double-valued argument and produces an int-valued result.
DoubleToLongFunction	Represents a function that accepts a double-valued argument and produces a long-valued result.
DoubleUnaryOperator	Represents an operation on a single `double`-valued operand that produces a `double`-valued result.
Function<T,R>	Represents a function that accepts one argument and produces a result.
IntBinaryOperator	Represents an operation upon two `int`-valued operands and producing an `int`-valued result.
IntConsumer	Represents an operation that accepts a single `int`-valued argument and returns no result.
IntFunction<R>	Represents a function that accepts an int-valued argument and produces a result.

Typowe interfejsy funkcyjne

Interfejs	Wejście	Wyjście
`Function<T, R>`	`T`	`R`
`Supplier<T>`	`T`	`void`
`Consumer<T>`	`void`	`T`
`Predicate<T>`	`T`	`boolean`

Producent

```
public class Game {
  private Player[] players = new Player[2];
  /* ... */
  public static Integer rollDice() {
    return (int) (Math.random() * 6) + 1;
  }
  public void play(Supplier<Integer> roll) {
    while(true)
      for(Player player : players)
        player.move(roll.get());
  }
}
Game game = new Game();
game.play(Game::rollDice);
```

Producent

```
int getRandomNumber()
{
    return 4;  // chosen by fair dice roll.
               // guaranteed to be random.
}
```

Źródło: xkcd.com

```
public class Game {
  private Player[] players = new Player[2];
  /* ... */
  public static Integer rollDice() {
    return (int) (Math.random() * 6) + 1;
  }
  public void play(Supplier<Integer> roll) {
    while(true)
      for(Player player : players)
        player.move(roll.get());


  }
}
Game game = new Game();
game.play(Game::getRandomNumber);
```

Predykat

```
public class Game {
  private Player[] players = new Player[2];
  public static final List<Integer> badPlaces
                                    = List.of(5, 10, 15);

  public static final boolean isInBadPlace(position) {
    return badPlaces.contains(position)
  }
}
```

Predykat

```java
public class Player {
  private int pos;
  private Predicate<Integer> isBad;

  public void setIsBad(Predicate<Integer> isBad) {
    this.isBad = isBad;
  }
  public void move(int delta) {
    this.pos += delta;
    if (isBad.test(this.pos)) {
      this.pos = 0;
    }
  }
}
```

```java
public class Game {
  private Player[] players = new Player[2];
  public static final List<Integer> badPlaces
                                    = List.of(5, 10, 15);

  public static final boolean isInBadPlace(position) {
    return badPlaces.contains(position)
  }
}

Player p1 = new Player(), p2 = new Player();
for (Player p : List.of(p1, p2))
  p.setIsBad(Game::isInBadPlace);
Game game = new Game(p1, p2);
```

Referencja do metody niestatycznej

```java
public class Player {
  private int pos;
  private Predicate<Integer> isBad;

  public void setIsBad(Predicate<Integer> isBad) {
    this.isBad = isBad;
  }
  public void move(int delta) {
    this.pos += delta;
    if (isBad.test(this.pos)) {
      this.pos = 0;
    }
  }
}
```

```java
public class Game {
  private Player[] players = new Player[2];
  public static final List<Integer> badPlaces
                                    = List.of(5, 10, 15);

  public static final boolean isInBadPlace(position) {
    return badPlaces.contains(position)
  }

  public Game(Player p1, Player p2) {
    players[0] = p1; players[1] = p2;
    for (Player p : List.of(p1, p2))
      p.setIsBad(badPlaces::contains);
  }
}
```

Konsument

```
public class FileCommander {
  public static void displayLength(String s) {
    System.out.printf("%s ", s.trim().length());
  }
  public static void cat(String filePath, Consumer<String> display) {
    try (BufferedReader fr = new BufferedReader(new FileReader(filePath))) {
      String line;
      while ((line = fr.readLine()) != null)
        display.accept(line);
    } catch (IOException e) {}
  }
}
FileCommander.cat("src/FileCommander.java", FileCommander::displayLength);
// 28 44 44 1 0 67 48 12 40 21 1 0...
```

Łączenie konsumentów

```
public class FileCommander {
  public static void displayLength(String s) { /* ... */ }
  public static void cat(String filePath, Consumer<String> display) {
    /* ... */
        display.accept(line);
    /* ... */
}
Consumer<String> lengthConsumer = FileCommander::displayLength;
Consumer<String> combined = lengthConsumer.andThen(System.out::println);
FileCommander.cat("src/FileCommander.java", combined);
// 28 public class FileCommander {
// 56 public static void displayLength(String s) { /* ... */ }
// ...
```

Własny interfejs funkcyjny

```
@FunctionalInterface
interface Replacer {
  String apply(String line, String target, String replacement);
}
public class FileCommander {
  public static void catWithReplacement(String fp, Replacer replacer, String target, String replacement) {
    try (BufferedReader br = new BufferedReader(new FileReader(fp))) {
      String line;
      while ((line = br.readLine()) != null) {
        line = replacer.apply(line, target, replacement);
        System.out.println(line)
      }
    } catch (IOException _) { }
  }
}
```

Anonimowa klasa wewnętrzna

```
public static void main(String[] args) {
  Replacer replacer = new Replacer() {
    @Override
    public String apply(String line, String target, String replacement) {
      return line.replace(target, replacement);
    }
  };


  catWithReplacement("src/FileCommander.java", replacer, "new", "old");
  // public static void main(String[] args) {
  //   Replacer replacer = old Replacer() {
  //   ...
}
```

Inna implementacja

```
public static void main(String[] args) {
  Replacer replacerCaseInsensitive = new Replacer() {
    @Override
    public String apply(String line, String target, String replacement) {
      Pattern p = Pattern.compile(Pattern.quote(target), Pattern.CASE_INSENSITIVE);
      Matcher m = p.matcher(line);
      return m.replaceAll(replacement);
    }
  };
  catWithReplacement("src/FileCommander.java", replacerCaseInsensitive, "NEW", "OLD");
  // public static void main(String[] args) {
  //   Replacer replacer = OLD Replacer() {
  //   ...
}
```

Funkcje lambda

```
public static String doNothing(String line) {
  return line;
}
public static void cat(String filePath) {
  cat(filePath, FileCommander::doNothing);
}
public static void cat(String filePath) {
  Function<String, String> doNothing = line -> line;
  cat(filePath, doNothing);
}
public static void cat(String filePath) {
  cat(filePath, line -> line);
}
public static void cat(String filePath) {
  cat(filePath, line -> { return line;});
}
```

Funkcje lambda implementujące interfejsy

Interfejs	Funkcja lambda
`Function<T, R>`	`input -> output`
`Supplier<T>`	`() -> output`
`Consumer<T>`	`input -> function(input)`
`Predicate<T>`	`(input) -> true lub (input) -> false`

Lambda producenta

```
public class Game {
  private Player[] players = new Player[2];
  /* ... */
  public void play(Supplier<Integer> roll) {
    while(true)
      for(Player player : players)
        player.move(roll.get());
  }
}
Game game = new Game();
game.play(() -> (int) (Math.random() * 6) + 1);
```

Lambda predykatu

```java
public class Player {
  private int pos;
  private Predicate<Integer> isBad;
  public void setIsBad(Predicate<Integer> isBad) {
    this.isBad = isBad;
  }
  public void move(int delta) {
    this.pos += delta;
    if (isBad.test(this.pos)) {
      this.pos = 0;
    }
  }
  public void getPos() { return pos; }
}
```

```java
public class Game {
  private Player[] players = new Player[2];

  public Game(Player p1, Player p2) {
    players[0] = p1; players[1] = p2;
    p1.setIsBad(pos -> pos == p2.getPos());
    p2.setIsBad(pos -> pos == p1.getPos());
  }
}
```

Przechwytywanie zmiennych

```java
public class Player {
  private int pos;
  private Predicate<Integer> isBad;
  public void setIsBad(Predicate<Integer> isBad) {
    this.isBad = isBad;
  }
  public void move(int delta) {
    this.pos += delta;
    if (isBad.test(this.pos)) {
      this.pos = 0;
    }
  }
  public void getPos() { return pos; }
}
```

```java
public class Game {
  private Player[] players = new Player[2];


  public Game(Player p1, Player p2) {
    players[0] = p1; players[1] = p2;
    p1.setIsBad(pos -> pos == p2.getPos());
    p2.setIsBad(pos -> pos == p1.getPos());
  }
}
```

Lambda konsumenta

```
public class FileCommander {
  public static void cat(String filePath, Consumer<String> display) {
    try (BufferedReader fr = new BufferedReader(new FileReader(filePath))) {
      String line;
      while ((line = fr.readLine()) != null)
        line = display.accept(line);
    } catch (IOException e) {}
  }
}

FileCommander.cat("src/FileCommander.java", s -> System.out.printf("%s ", s.trim().length()));
```

Problem – numerowanie linii

```
public static void catWithLineNumbers(String filePath) {
  try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
    String line;
    int number = 1;
    while ((line = br.readLine()) != null)
      System.out.println(String.format("%4d | %s", number++, line));
    }
}
FileCommander.catWithLineNumbers("src/Main.java");

// 1 | public class Main {
// 2 | public static void main(String[] args) {
// 3 | FileCommander.catWithLineNumbers("src/Main.java");
```

Problem – numerowanie linii

```
public class FileCommander {
  public static void cat(String filePath, Function<String, String> action) { /* ... */ }

  public static void catWithLineNumbers(String filePath) throws IOException {
    int number = 1;
    Function<String, String> modifier = line -> String.format("%4d | %s", number++, line);
    cat(filePath, modifier);
  }
}
```

Lambda przechwytuje tylko zmienne finalne

```
public class FileCommander {
  public static void cat(String filePath, Function<String, String> action) { /* ... */ }

  public static void catWithLineNumbers(String filePath) throws IOException {
    AtomicInteger i  = new AtomicInteger(1);
    Function<String, String> modifier = line -> String.format("%4d | %s", i.getAndIncrement(), line);
    cat(filePath, modifier);
  }
}
```

Strumienie

Lista plików, konsument

```
public static void cat(List<String> filePaths) {
  for (String filePath: filePaths)
    cat(filePath);
}
```

```
public static void cat(List<String> filePaths) {
  Consumer<String> cat = FileCommander::cat;
  filePaths.forEach(cat);
}
```

```
FileCommander.cat(List.of("src/Main.java", "src/FileCommander.java"));
```

Stream

```java
public static void cat(List<String> filePaths) {
  filePaths.forEach(FileCommander::cat);
}
```

```java
public static void cat(List<String> filePaths) {
  Stream<String> filePathsStream = filePaths.stream();
  filePathsStream.forEach(FileCommander::cat);
}
```

```java
FileCommander.cat(List.of("src/Main.java", "src/FileCommander.java"));
```

Predykat i filtrowanie

```
public static void catWithFilter(List<String> filePaths, String substr) {
  Predicate<String> contains = str -> str.contains(substr);
  Stream<String> filePathsStream = filePaths.stream();
  filePathsStream = filePathsStream.filter(contains);
  filePathsStream.forEach(FileCommander::cat);
}

public static void catWithFilter(List<String> filePaths, String substr) {
  filePaths.stream()
          .filter(str -> str.contains(substr))
          .forEach(FileCommander::cat);
}

FileCommander.catWithFilter(List.of("src/Main.java", "src/FileCommander.java", ".gitignore"), "java");
```

Obiekt klasy FileCommander

```java
public class FileCommander {
  private Path path;

  public FileCommander() {
    this.path = Paths.get("");
  }

  public void home() {
    this.path = Path.of(System.getProperty("user.home"));
  }

  public void cd(String path) {
    this.path = this.path.resolve(path).normalize();
  }

  public String pwd() {
    return path.toAbsolutePath().toString();
  }

}
```

Strumień ścieżek

```java
public void ls() throws IOException {
  try (Stream<Path> stream = Files.list(path)) {
    stream.forEach(System.out::println);
  } catch (IOException e) {
    throw new
      IOException("Cannot read: " + path, e);
  }
}
```

```java
try {
    FileCommander fileCommander =
                                new FileCommander();
    fileCommander.cd("src");
    fileCommander.ls();
}
catch (IOException e) {
    e.printStackTrace();
}

// src/Main.java
// src/FileCommander.java
```

Mapowanie

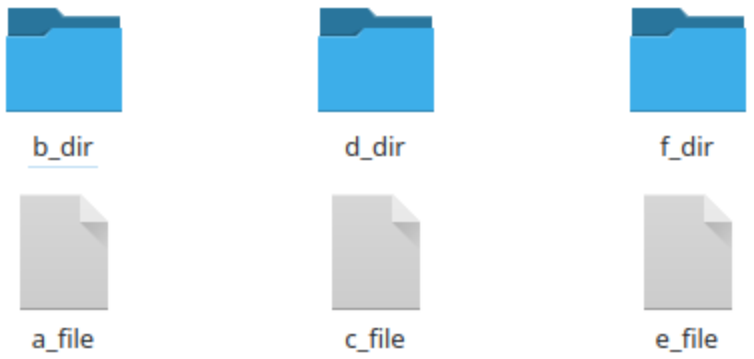
```java
public void ls() throws IOException {
  try (Stream<Path> stream = Files.list(path)) {
    stream
        .map(Path::getFileName)
        .forEach(System.out::println);
  } catch (IOException e) {
    throw new
      IOException("Cannot read: " + path, e);
  }
}
```

```java
try {
    FileCommander fileCommander =
                                new FileCommander();
    fileCommander.cd("src");
    fileCommander.ls();
}
catch (IOException e) {
    e.printStackTrace();
}

// Main.java
// FileCommander.java
```

Sortowanie

```java
public void ls() throws IOException {
  try (Stream<Path> stream = Files.list(path)) {
    stream
        .map(Path::getFileName)
        .forEach(System.out::println);
  } catch (IOException e) {
    throw new
      IOException("Cannot read: " + path, e);
  }
}
```

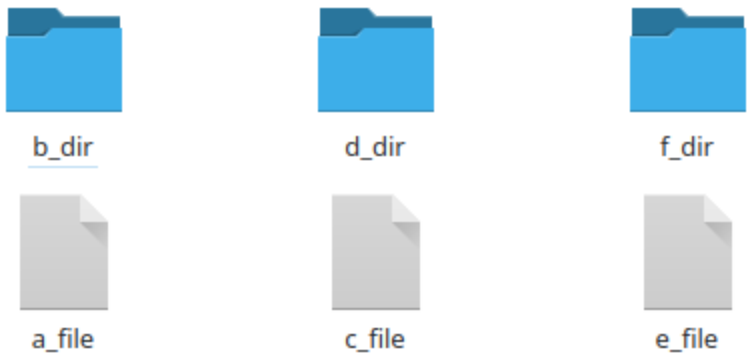


Wyjście:

c_file e_file b_dir d_dir f_dir a_file

Sortowanie domyślne

```java
public void ls() throws IOException {
  try (Stream<Path> stream = Files.list(path)) {
    stream
        .map(Path::getFileName)
        .sorted()
        .forEach(System.out::println);
  } catch (IOException e) {
    throw new
      IOException("Cannot read: " + path, e);
  }
}
```

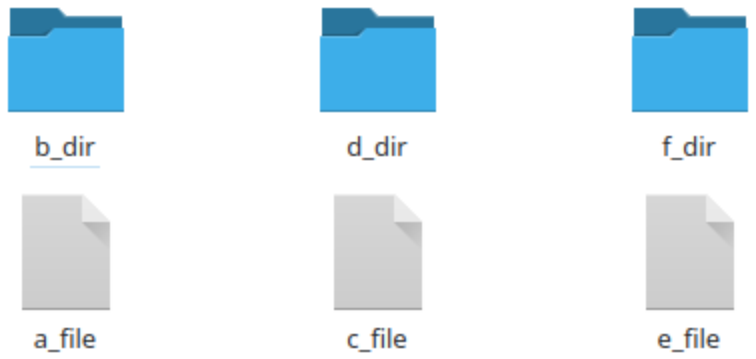


Wyjście:

a_file b_dir c_file d_dir e_file f_dir

Komparator

```
try (Stream<Path> stream = Files.list(path)) {
  Comparator<Path> comparator =
    (p1, p2) -> Boolean.compare(
      !Files.isDirectory(p1),
      !Files.isDirectory(p2)
    );
  stream
     .sorted(comparator)
     .map(Path::getFileName)
     .forEach(System.out::println);
} catch (IOException e) {
  throw new IOException("Cannot read: " + path,
e);
}
```



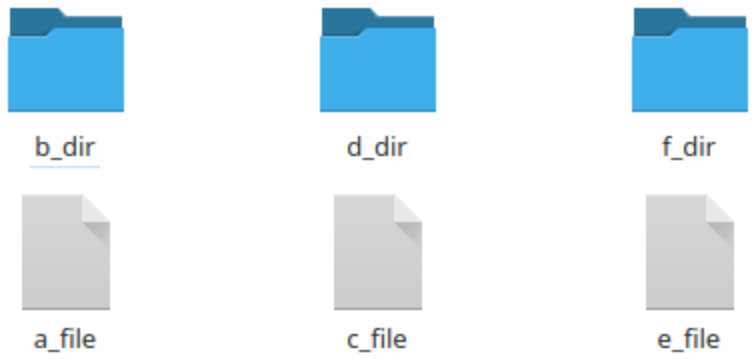
Wyjście:

b_dir d_dir f_dir c_file e_file a_file

Łączenie porównań

```
try (Stream<Path> stream = Files.list(path)) {
  Comparator<Path> comparator =
    (p1, p2) -> Boolean.compare(
      !Files.isDirectory(p1),
      !Files.isDirectory(p2)
    );
  comparator =
  comparator.thenComparing(Path::getFileName);

  stream
      .sorted(comparator)
      .map(Path::getFileName)
      .forEach(System.out::println);
} catch (IOException e) {
  throw new IOException("Cannot read: " + path,
e);
}
```

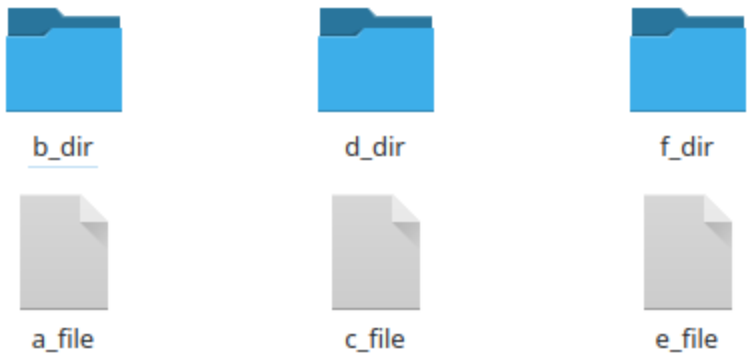


Wyjście:

b_dir d_dir f_dir a_file c_file e_file

Rozbudowana lambda - zła praktyka

```
private static final String RESET = "\033[0m";
public static final String BLUE = "\033[0;34m";
/* ... */
  stream
      .sorted(comparator)
      .map(path -> {
        String pathString =
          path.getFileName().toString();
        if (Files.isDirectory(path))
          return BLUE + pathString + RESET;
        else return pathString;
      })
      .forEach(System.out::println);
```



Wyjście:

b_dir d_dir f_dir a_file c_file e_file

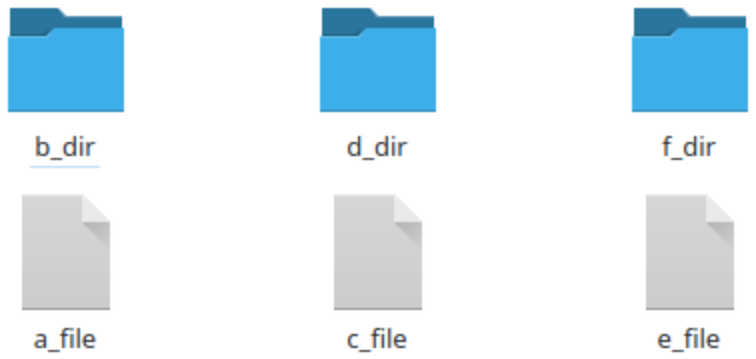
Podział mapowania

```
private static final String RESET = "\033[0m";

public static final String BLUE = "\033[0;34m";

/* ... */

  stream
      .sorted(comparator)
      .map(path -> new Path[]
                      {path,path.getFileName()})
      .map(array -> Files.isDirectory(array[0])
            ? BLUE + array[1] + RESET : array[1])
      .forEach(System.out::println);
```



Wyjście:

b_dir d_dir f_dir a_file c_file e_file

Zwrócenie wszystkich linii z pliku

```
public static List<String> read(String filePath) {
    List<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        String line;
        while ((line = br.readLine()) != null) {
            lines.add(line);
        }
    }
    catch(IOException e) {};
    return lines;
}
```

```
for (String line: FileCommander.read("src/Main.java"))
      System.out.println(line);
```

Strumień linii, kolektor

```java
public static List<String> read(Path path) {
    try (Stream<String> stream = Files.lines(path);) {
        return stream.collect(Collectors.toList());
    } catch (IOException e) {
        return List.of();
    }
}


FileCommander.read(Paths.get("src/Main.java"))
  .forEach(System.out::println);
```

Filtrowanie pliku

```
public static List<String> find(Path path, String text) {
    return read(path).stream()
            .filter(line -> line.contains(text))
            .collect(Collectors.toList());
}
```

```
FileCommander.find(Paths.get("src/Main.java"), "fileCommander").forEach(System.out::println);
// FileCommander.find(Paths.get("src/Main.java"), "fileCommander").forEach(System.out::println);
```

Podświetlenie wyników

```java
private static final String RESET = "\033[0m";
public static final String RED_BOLD = "\033[1;31m";


public static String highlight(String line, String text) {
    StringBuilder sb = new StringBuilder();
    int lastIndex = 0;
    int index = line.indexOf(text);
    while (index != -1) {
        sb.append(line.substring(lastIndex, index));
        sb.append(RED_BOLD);
        sb.append(text);
        sb.append(RESET);
        lastIndex = index + text.length();
        index = line.indexOf(text, lastIndex);
    }
    sb.append(line.substring(lastIndex));
    return sb.toString();
}
```

Strumień z tablicy, łączenie do napisu

```
public static String highlight(String line, String text) {
      return Arrays.stream(line.split(text))
              .map(part -> part)
              .collect(Collectors.joining(String.format("%s%s%s", RED_BOLD, text, RESET)));
  }


System.out.println(FileCommander.highlight("Ala ma kota, a kot ma koguta.", "ko"));

// Ala ma kota, a kot ma koguta.
```

Filtrowanie i podświetlanie

```
public static List<String> find(Path path, String text, boolean highlight) {
    return read(path).stream()
            .filter(line -> line.contains(text))
            .map(highlight
                    ? line -> FileCommander.highlight(line, text)
                    : line -> line
            )
            .collect(Collectors.toList());
}

FileCommander.find(Paths.get("src/Main.java"), "fileCommander", true)
    .forEach(System.out::println);

// FileCommander.find(Paths.get("src/Main.java"), "fileCommander", true)
    .forEach(System.out::println);
```

Spłaszczanie mapowania

```
public List<String> find(String text, boolean highlight) {
    try (Stream<Path> stream = Files.walk(path)) {
        return stream
                .filter(filePath -> !Files.isDirectory(filePath))
                .flatMap(filePath -> FileCommander.find(filePath, text, highlight).stream())
                .collect(Collectors.toList());
    } catch (IOException e) { return List.of(); }
}

FileCommander fileCommander = new FileCommander();

fileCommander.cd("src/");

fileCommander.find("public", true).forEach(System.out::println);

// public class Main {

// public class FileCommander {

// public List<String> find(String text, boolean highlight) {

// ...
```